

Abbiamo un dizionario personalizzato, che è un array molto grande (10^6 slot) inizialmente vuoti.

Ogni nuova parola viene cercata nel dizionario per vedere se c'è da aggiungere, e se sì, è inserita riordinando l'array.

PROBLEMA: L'array è molto grande e (inizialmente) molto vuoto!

C'è un modo di risolvere questo problema, con il bonus che, se "d" sta in posizione "i", allora lo posso trovare non in $O(\log(n))$, ma in $O(\log(i))$.

L'idea è usare una **RICERCA ESPONENZIALE**:

POSIZIONE: $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32$.

Supponiamo che $A[32] > d$. Allora sappiamo che "d" è compreso tra la posizione 16 e la posizione 32.

Un altro modo di gestire queste strutture è usare le **TABELLE DI HASH**.

Le **TABELLE DI HASH** sono vettori che permettono di recuperare delle informazioni velocemente tramite delle caratteristiche degli elementi

$$h: U \rightarrow [0, n-1], \text{ con } n \ll |U|$$

Fondamentalmente, h deve essere:

- Facile da calcolare
 - Deve distribuire in modo uniforme le collisioni.
- Abbiamo una **COLLISIONE** quando $h(k_1) = h(k_2)$ con $k_1 \neq k_2$

ESEMPIO: Se k è un intero, $h(k) = k \bmod n$ è una semplice funzione di hash: è facile da calcolare

Un altro esempio può essere il cambio di base.

NOTA: k è la chiave, non l'oggetto stesso del dizionario

Tipicamente, un modo di gestire le collisioni è usare delle tecniche di **ISPEZIONE**. Vediamole in un esercizio...

② $m = 11$, chiavi: 35, 83, 57, 26, 15, 63, 97, 46

$$h(k) = k \bmod 11$$

Creare H con le chiavi date usando:

(a) Tecnica di scansione **LINEARE**

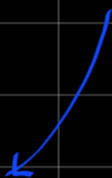
(b) // // **DOPPIA** con $h_1(k) = 1 + (k \bmod 10)$

(c) // // **QUADRATICA**

SOLUZIONE:

scriviamo le varie hash
delle chiavi

k	$h(k)$
35	2
83	6
57	2
26	4
15	4
63	8
97	9
46	2



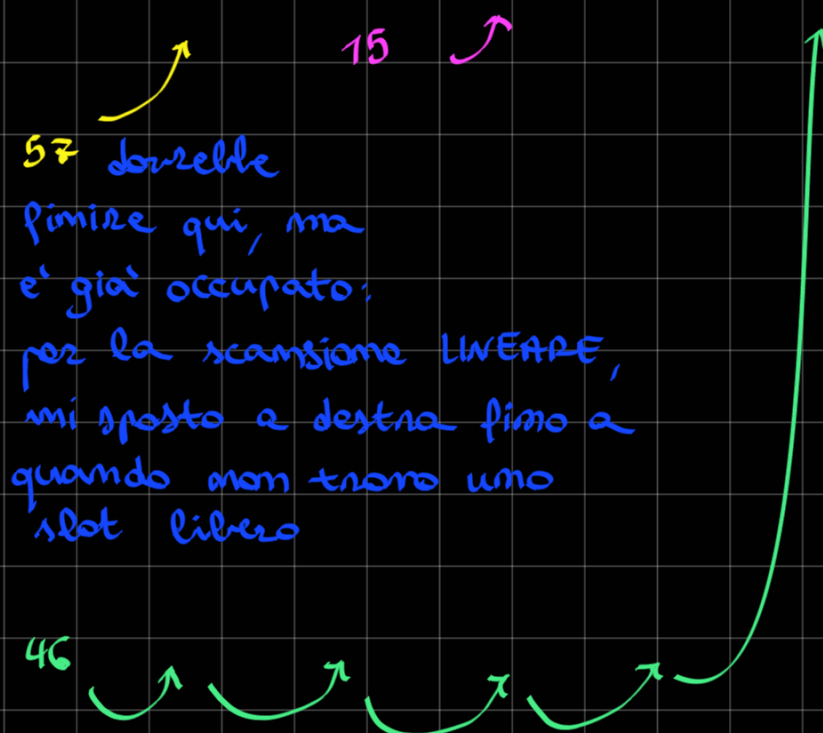
disegniamo una tabella con $m = 11$ "bucket"...

0 1 2 3 4 5 6 7 8 9 10

--	--	--	--	--	--	--	--	--	--	--

(a) SCANSIONE **LINEARE**

0	1	2	3	4	5	6	7	8	9	10
		35	57	26	15	83	46	63	97	



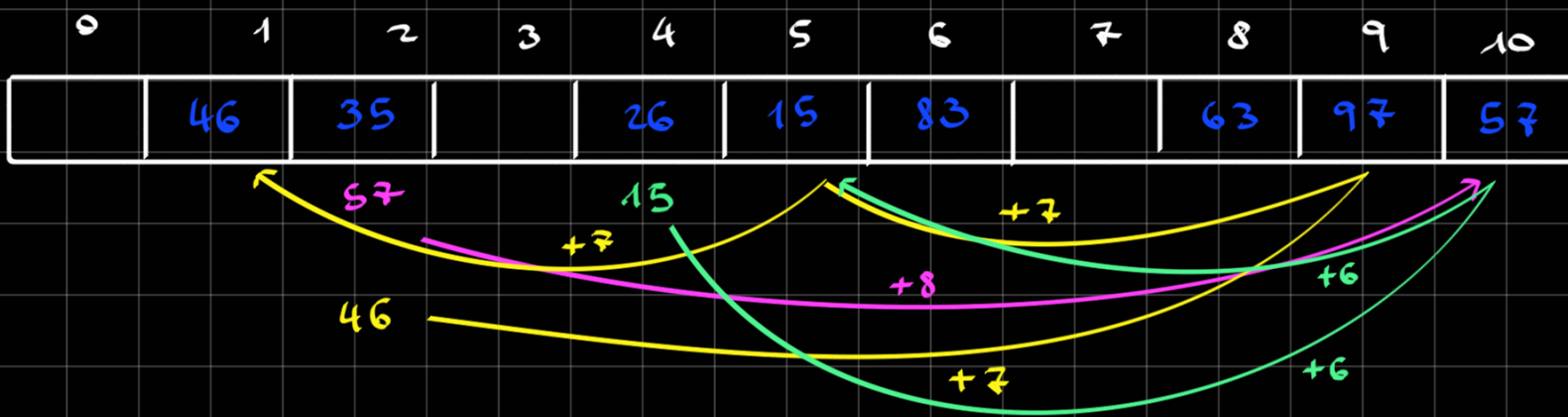
ESEMPIO: Voglio cercare 68. Come faccio?

- $h_1(68) = 2$
 - Controllo la tabella in indice 2. C'è "35"
 - "68" potrebbe essere stato inserito in un altro slot della tabella $\rightarrow$ faccio la scansione lineare
 - Continuo fino a quando non trovo uno slot vuoto: in questo caso concludiamo che 68 non è nella tabella
- $\rightarrow$ costo della ricerca = 0 (numero di chiavi inserite)

(b) SCANSIONE **DOPIA** con $h_1(k) = 1 + (k \bmod 10)$.
Cioè: il passo è costante per ogni chiave, ma è diverso per tutte le chiavi

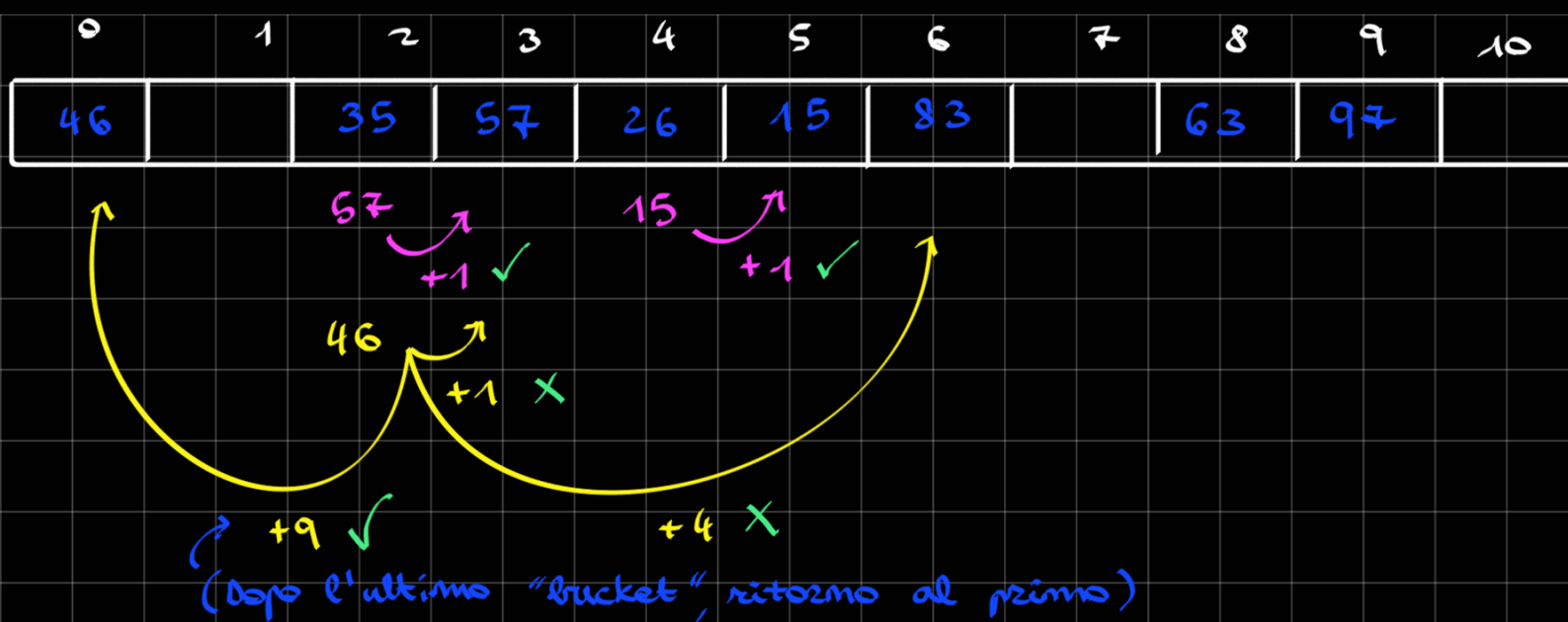
calcoliamo le varie hash che ci serviranno per la scansione e ridisegniamo la tavola. Il numero di passi fatto per ogni el. che va in collisione è dettato dal suo h_1 . Quindi, se 83 va in collisione, devo fare $h_1(83) = 4$ passi

k	$h_1(k)$
35	6
83	4
57	8
26	7
15	6
63	4
97	8
46	7



Cercando "68", inizierò da 2 e mi sposterò di $1 + 68 \bmod 10 = 9$ celle ogni volta...

(c) SCANSIONE QUADRATICA: in questo caso, a ogni cella vuota, aumento quadraticamente il numero di passi fatti



la cosa più difficile che si può fare con le tavole hash è la cancellazione di un elemento: non basta rimuovere l'oggetto, perché bisogna assicurarsi che tutti gli altri elementi stiano nella posizione giusta

Ho due strade:

- Cancello la tabella e rimetto tutti gli elementi (lavorioso)
- Metto un valore speciale che significa: "questa cella è libera per metterci qualcosa, ma se stai cercando un elemento considera la cella come se fosse occupata, in questo modo continui a scansionare". Questo valore speciale è chiamato **TOMBSTONE**

2

Si consideri il caso di una tabella di hash di $m = 8$ elementi, le cui chiavi sono costituite da numeri interi, gestita con la disciplina dell'indirizzamento aperto.

1 Si mostri lo stato della tabella, dopo ognuna delle seguenti operazioni

- inserimento di 2
- inserimento di 10
- inserimento di 18
- cancellazione di 10
- inserimento di 26

considerando $h(k, i) = (k + i) \bmod 8$.

SOLUZIONE:

Calcoliamo l'hash dei vari elementi.

k	$h(k)$
2	2
10	2
18	2
26	2

Osserviamo che l'ispezione in questo caso è LINEARE, ma dobbiamo fare attenzione a cosa succede quando cancello un elemento

• Inserisco 2

	2						
--	---	--	--	--	--	--	--

• Inserisco 10 $\Rightarrow$ Collisione $\Rightarrow$ Scansione LINEARE

	2	10					
--	---	----	--	--	--	--	--

10 

• Inserisco 18 $\Rightarrow$ Collisione $\Rightarrow$ Scansione LINEARE

	2	10	18				
--	---	----	----	--	--	--	--

18

- Cancello 10 : metto una TOMBSTONE in modo che, se voglio cercare 18, la scansione vada a buon fine...

	2	X	18				
--	---	---	----	--	--	--	--

CERCO
10

- Inserisco 26 $\Rightarrow$ COLLISIONE $\Rightarrow$ scansione $\Rightarrow$
Posso inserire nel bucket in cui si trova il tombstone

	2	26	18				
--	---	----	----	--	--	--	--

26

3

Si consideri una tabella hash di $m = 1024$ elementi, organizzata con indirizzamento aperto. La funzione di hash utilizzata è $h(k) = 4k \bmod 1024$, la sequenza di ispezione è $h'(k, i) = (h(k) + 2i) \bmod 1024 = (4k + 2i) \bmod 1024$.

- 1 Qual è il numero massimo di elementi inseribili nella tabella?
- 2 Qual è il numero medio di tentativi per accedere ad un elemento dopo l'inserimento di 256 elementi?

* ISPEZIONE LINEARE CON COEFFICIENTE 2

SOLUZIONE:

Notiamo che, essendo $1024 = 2^{10}$ e $4 = 2^2$, la nostra funzione di hash andrà a inserire gli elementi solo nelle celle che sono multiple di 4!

Inoltre, l'ispezione utilizzata mi farà "saltare" sempre di 2 caselle $\Rightarrow$ Non c'è modo di finire nelle celle dispari! $\Rightarrow$ al massimo posso inserire $\frac{1024}{2} = 512$ elementi nella tabella!

Questo dimostra l'importanza di avere una "buona" funzione di hash, ed è per questo che non si sceglie quasi mai un coefficiente pari per la funzione di scansione.

Sappiamo che il tempo medio all'inserimento di un elemento nella tabella di hash è $\frac{1}{\alpha} \ln \left(\frac{1}{1-\alpha} \right)$,
dove $\alpha = \frac{n}{m} = \frac{\text{\# ELEM. IN TABELLA}}{\text{\# CELLE DELLA TABELLA}}$

FATTORE DI CARICO

Il problema ci dice che $n = 256$, mentre $m = 512$ (occhio: poiché metà delle celle non sono raggiungibili, è come se non esistessero!)

Quindi $\alpha = \frac{1}{2} \Rightarrow \frac{1}{\alpha} \ln \left(\frac{1}{1-\alpha} \right) \approx 1,38$ passi

vediamo dei problemi in cui ha senso usare le tabelle di hash...

- 4) Dato un array A di n elementi e $d \in \mathbb{Z}$, trovare, se esistono, $1 \leq i < j \leq n$ tali che $A[i] + A[j] = d$
-

SOLUZIONE:

Abbiamo già visto come risolvere questo problema utilizzando l'ordinamento dell'array. Ma possiamo risolvere senza ordinare l'array?

RISPOSTA: Sì, e in tempo medio LINEARE facendo questa osservazione:

Mentre scorro l'array, se l'elemento corrente è x , dobbiamo cercare $d - x$

→ Al posto di cercarlo nell'array, lo cerco in una tabella di hash inizialmente vuota. Se non lo trovo, aggiungo l'elemento x nella tabella e vado al prossimo elemento dell'array

COMPLESSITÀ: $O(n)$

- scorro tutti gli elementi dell'array (lineare)
- per ogni elemento dell'array, cerco nella tabella di hash (tempo costante)

Scriviamo lo pseudocodice...

CercaCoppia (A, n, d):

inizializza H : tabella hash vuota

per i da 1 a n :

 corrente = $A[i]$

 compl = $d - \text{corrente}$

 se cerca(H, compl):

 return True

 else

 Inserisci($H, \text{corrente}$)

return False

-
- 5) Dato l'array A di n interi, trovare la lunghezza della più lunga sottosequenza di numeri consecutivi in A
cioè: trovare il massimo $k \in \mathbb{N}$ t.c.
 $\exists i : i, i+1, \dots, i+k \in A$

ESEMPIO: $A = [100, 4, 20, 1, 3, 2] \rightarrow k = 4$

SOLUZIONE:

1° METODO: Ordino l'array A e scorro tutto l'array cercando la sequenza più lunga
(* ESERCIZIO: scrivi lo pseudocodice)

2° METODO:

- Inserisco tutti gli elementi di A in una HT (chiamiamola H)

- per ogni elemento di A , verifico se è l'inizio di una sottosequenza di elementi consecutivi (eventualmente con $k=1$), cioè se $(A[i]-1) \notin A$. lo faccio cercando $A[i]-1$ in H
- Se $A[i]$ è l'inizio di una sottosequenza, inizio a contare i numeri consecutivi ad $A[i]$ e li cerco in H

PSEUDOCODICE:

sottosequenza(A, n):

crea H : una HT vuota

per i da 1 a n :

inserisci $A[i]$ in H

massimo = 0

per i da 1 a n :

corrente = $A[i]$

se corrente - 1 $\notin H$ *

maxcorrente = 1

while corrente + maxcorrente $\in H$

maxcorrente ++

se maxcorrente > max

max = maxcorrente

return max

la cosa strana di questo algoritmo è la sua complessità:

all'apparenza, sembra quadratica (un for-loop e un while-loop interno)

Tuttavia, la riga * garantisce che il while-loop sia eseguito solo quando troviamo l'inizio della sottosequenza

Praticamente, è come se i due cicli non fossero annidati $\rightarrow$ COMPLESSITÀ = $O(n)$. Senza , la complessità è $O(n^2)$

- 6 Sono date due stringhe, s_1 e s_2 , di lunghezza rispettivamente n_1 e n_2 caratteri: stabilire se s_2 è un anagramma di s_1
-

SOLUZIONE:

1° MODO: Ricordando che i caratteri si possono ordinare tramite la codifica ASCII, un primo metodo consiste nell'ordinare l'array e stabilire se i due array ordinati sono uguali $\rightarrow$ complessità: $O(n \log n)$

↑
per essere uguali, $n_1 = n_2 = n$

2° MODO: costruisco l'array delle frequenze, e verifico che l'array è uguale per tutti $\rightarrow$ complessità: $O(n^2)$

VARIANTE:

Sono dati due vettori, $\underline{u}, \underline{v} \in \mathbb{Z}^n$, stabilire se $\underline{u}$ è un anagramma di $\underline{v}$

SOLUZIONE:

si possono usare le tabelle di hash in modo furbo.

IDEA: uso una HT per tenere traccia delle frequenze dei coefficienti di $\underline{u}$ e $\underline{v}$.

A questo punto, scorro gli elementi di $\underline{u}$: per ogni coefficiente, calcolo il suo hash e lo cerco in H

In H salvo i coefficienti in una struct con VALORE e FREQUENZA. Ogni cella di H è un puntatore a una di queste struct.

- Se non trovo il valore in tabella, inserisco un nuovo nodo con il VALORE corrispondente e FREQUENZA = 1
- Se lo trovo, incremento di 1 la FREQUENZA

In questo modo, in poco spazio riusciamo a conservare tutta l'informazione del vettore $\underline{u}$ (questo è il "potere" che rende le HT molto amate!)

A questo punto, basta scorrere il vettore $\underline{v}$ e vedere se le occorrenze coincidono:

- Se trovo, decremento il campo FREQUENZA
- Se non trovo, o un campo FREQUENZA diventa negativo $\rightarrow$ False
- Alla fine, controllo che tutti i campi frequenza siano zero. In tale caso $\rightarrow$ True

(notiamo che devo scorrere la HT, che si fa in $O(n)$)

si verifica che COMPLESSITA' MEDIA: $O(n)$
